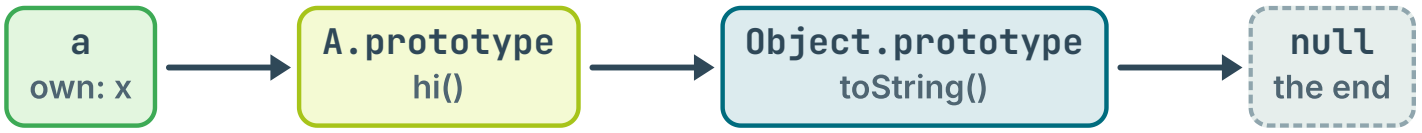
Prototypes & Classes

Objects inherit through a chain of prototypes

Property lookup climbs the chain



class adds: needs new | TDZ | #private | static | hidden from for...in

1 Own vs shared

```
class A {
  static kind = "A";
  x = 5;
  hi() { return 1; }
}
const a = new A();
Object.hasOwn(a, "x");  // true
Object.hasOwn(a, "hi"); // false
```

2 extends & super

```
class B extends A {
  constructor() { super(); }
  hi() { return super.hi() + 1; }
}
new B().hi();          // 2
B.kind;                // "A"
new B() instanceof A; // true
```

TRAP What does this log?

```
const p = { n: 1 };
const a = Object.create(p);
a.n = 2;
console.log(p.n, a.n);
```

1 2

a.n = 2 creates an own property. The prototype stays untouched.

KEY TAKEAWAY

- **chain** — Lookup climbs up to null
- **class** — Same links, plus extras
- **own** — Writes stay on the object

? Follow-up questions to expect

- **Is `class` just syntax over prototypes?** → Mostly. But it needs new, has #private and TDZ.
- **What does `super()` do?** → Runs the parent constructor. Required before this.
- **What does `Object.create` do?** → Makes an object with the given prototype.
- **Own vs inherited: how do you tell?** → Object.hasOwn(obj, "key").